

SE4030 — Secure Software Development

CraveDrop Security Assessment

and Secure OpenID Connect Customer Login

Assignment report

Hansaja A. M. G	IT22171856
Dharmasiri I. D. N. D	IT22254320
Sanjeewa P. D. L. B	IT22629708
Aluthwaththa A. W. D. S. M	IT22267290

September 2026

Contents

1	Executive summary	1
2	Assignment alignment and project eligibility	1
3	Application scope and assessment method	2
3.1	Architecture and trust boundaries	2
3.2	Method	2
3.3	Scope limits	2
4	Finding summary	2
4.1	V1: Missing authentication on payment actions	3
4.2	V2: Order insecure direct object reference	3
4.3	V3: Order mass assignment	3
4.4	V4: Unauthorized delivery mutation	4
4.5	V5: Broken driver administration authorization	4
4.6	V6: Payment amount and state manipulation	4
4.7	V7: Sensitive token disclosure and session design	5
5	Verification and reproducibility	5
5.1	Fixtures and baseline integrity	5
5.2	Test evidence	5
5.3	Supporting tools	5
6	WSO2 OpenID Connect customer login	6
6.1	Feature objective	6
6.2	Security design	6
6.3	Feature verification	6
7	Remaining vulnerabilities and limitations	6
8	Preventive secure software engineering practices	7
9	Individual contributions	7
10	Conclusion	8

1 Executive summary

This report presents a security assessment of CraveDrop, a food-ordering and delivery application implemented as microservices. The work preserves the original vulnerable source at immutable tag [baseline-vulnerable](#), which resolves to commit [cb68a377f5b8cdc3b12f86883ac6fb703ef5e405](#). The baseline was not altered.

The team identified, reproduced, fixed, and regression-tested seven distinct vulnerabilities. The findings cover missing authentication, object-level authorization, mass assignment, function-level authorization, payment workflow integrity, and token disclosure. Each counted finding has a different root cause. The evidence includes a redacted live baseline request, a focused code change, a blocked-attack check, and a legitimate authorized control.

The project also adds one customer-login feature using WSO2 Identity Server 7.1.0. It uses OpenID Connect Authorization Code flow with S256 PKCE. The application validates the identity token server-side and issues its own strict HttpOnly session cookies. This feature is separate from the seven counted vulnerabilities.

All fixtures are synthetic. No secrets, live customer data, bearer tokens, cookies, or unredacted logs are included in the repository or this report.

2 Assignment alignment and project eligibility

The assignment requires an application with sufficient scope, at least seven distinct vulnerabilities, fixes, discussion of remaining risks and preventive practices, an OAuth or OpenID Connect feature, and evidence of individual contribution. This report addresses those requirements as follows.

Assignment requirement	Evidence in this submission
Application scope and suitability	CraveDrop has separate user, order, payment, delivery, driver, restaurant, notification, email, and SMS components. It is a food-ordering application, not an intentionally vulnerable learning application.
Seven distinct vulnerabilities	Sections 4 and 5 describe V1–V7, their distinct root causes, before evidence, fixes, and controls.
OAuth or OpenID Connect feature	Section 6 documents a WSO2 OIDC customer login using Authorization Code with S256 PKCE.
Discussion and unfixed risks	Sections 7 and 8 list remaining vulnerabilities, reasons, and preventive practices.
Individual contribution	Section 9 maps allocated work to commits, tests, evidence, and the planned presentation segments.

The original project is [nmdra/CraveDrop](#) and uses the MIT licence. The selected default-branch baseline is dated 17 July 2025, before the July 2026 semester start. The modified repository is [nmdra/CraveDrop-Secure-SSD](#). The eligibility record is in [UPSTREAM.md](#) and [docs/eligibility-evidence.md](#).

3 Application scope and assessment method

3.1 Architecture and trust boundaries

CraveDrop exposes a browser frontend through an NGINX gateway. The user service uses PostgreSQL. The order, payment, delivery, and driver services use separate service boundaries and databases. RabbitMQ supports asynchronous notification work. The assessment does not assume that gateway routing supplies authentication. Each changed controller or route checks identity and authorization at the relevant service boundary.

The principal types used in the tests are Customer A, Customer B, the assigned driver, and an unrelated driver. This makes the authorization checks reproducible without real personal data.

3.2 Method

The team used the following evidence-first procedure.

1. Preserve the upstream source at [baseline-vulnerable](#).
2. Create deterministic synthetic A/B customer, driver, restaurant, product, order, and delivery fixtures.
3. Run a redacted live HTTP attack against the baseline.
4. Apply the smallest focused fix that preserves a valid authorized path.
5. Run a negative regression test and a positive authorized control.
6. Link the evidence, test, and fix commit in [docs/finding-matrix.md](#).

A 401 or 403 response alone does not establish a complete fix. Each test also proves that an authorized synthetic principal can use the protected function.

3.3 Scope limits

The assessment counts only V1–V7. Delivery and notification BOLA variants were retained as backup observations, not counted as extra findings. Dependency advisories from [npmaudit](#) are also not counted without a reproduced application impact. This avoids reporting multiple URLs with the same root cause as separate vulnerabilities.

4 Finding summary

ID	Finding	Classification	Verified secure result
V1	Missing payment authentication	CWE-306; OWASP API1	Anonymous payment actions return 401; an authenticated customer reaches the payment-start handler.
V2	Order IDOR/BOLA	CWE-639; OWASP API1	A foreign customer cannot read the other customer's order; the owner can.
V3	Order mass assignment	CWE-915; OWASP API3	Client attempts to set protected order fields return 400; valid authorized actions work.
V4	Unauthorized delivery mutation	CWE-862; OWASP API5	Only the assigned driver can make allowed status and location updates.
V5	Broken driver administration authorization	CWE-862; OWASP API5	A driver can update own availability, not another driver's.

ID	Finding	Classification	Verified secure result
V6	Payment amount and state manipulation	CWE-841; API business logic	The server calculates the amount and creates a pending order despite tampered values.
V7	Sensitive token disclosure and session design	CWE-922, CWE-532; OWASP API2	No bearer token is returned in JSON, stored in browser storage, or logged.

4.1 V1: Missing authentication on payment actions

Baseline reproduction. An anonymous request to [POST/api/payments/create-payment-intent](#) reached the payment handler. It did not receive an authentication error. The redacted live response is [evidence/V1-before.txt](#).

Impact. An unauthenticated actor could invoke a payment action, consume payment-provider resources, and bypass the intended customer boundary.

Fix. Payment routes now require a verified Bearer JWT. The service derives the caller identity from the verified token rather than request input.

Regression controls. The same anonymous action returns 401. An authenticated synthetic customer reaches the payment-start handler. The local Stripe test configuration rejects the request after authorization, which proves the authorization boundary but not a real card charge. See [evidence/V1-after.txt](#), [security-tests/runtime-v1-v3.mjs](#), commits [4904430](#) and [2e7d8bf](#).

Residual risk. Stripe configuration and provider-side controls remain necessary. This assessment does not process a live card charge.

4.2 V2: Order insecure direct object reference

Baseline reproduction. An unauthenticated request read Customer A's synthetic order by order ID. The request returned the order record. See [evidence/V2-before.txt](#).

Impact. An attacker who knows or obtains an order ID could read or modify another customer's order.

Fix. Order reads and mutations now require an authenticated customer. The controller scopes the order query to both the requested order ID and the verified customer ID.

Regression controls. Customer B receives 404 when requesting Customer A's order. Customer A can read the owned order. Evidence is [evidence/V2-after.txt](#); the runtime control is [security-tests/runtime-v1-v3.mjs](#); the commits are [4904430](#) and [2e7d8bf](#).

Residual risk. The order ID remains an identifier. Every future order route must preserve the ownership scope.

4.3 V3: Order mass assignment

Baseline reproduction. An unauthenticated order update supplied [userId](#), [totalAmount](#), [status](#), and [createdAt](#). The baseline persisted these client-controlled fields. See [evidence/V3-before.txt](#).

Impact. A caller could alter ownership, price, lifecycle state, or audit data.

Fix. The controller rejects protected fields and allow-lists only supported customer-editable fields. The server owns order identity, price, payment state, and timestamps.

Regression controls. A payload containing protected fields returns 400. A valid authorized update and order creation succeed. See [evidence/V3-after.txt](#), [security-tests/runtime-v1-v3.mjs](#), commits [4904430](#) and [2e7d8bf](#).

Residual risk. A new editable field must undergo explicit review before it is added to the allow-list.

4.4 V4: Unauthorized delivery mutation

Baseline reproduction. An anonymous request changed a delivery status, and the same boundary accepted driver-location changes. The baseline evidence is [evidence/V4-before.txt](#).

Impact. An attacker could falsify delivery progress or tracking data.

Fix. Delivery writes require a valid driver JWT, an assigned-driver identity match, and an allow-listed status transition.

Regression controls. Anonymous, customer, and unrelated-driver writes fail. The assigned driver can make the demonstrated valid status and location updates. A reverse transition returns 409. See [evidence/V4-after.txt](#), [security-tests/runtime-v4-v7.mjs](#), [security-tests/regression-v4-v7.mjs](#), commits [2f58bbf](#) and [8505d31](#).

Residual risk. The service does not prove the integrity of a driver device or GPS source.

4.5 V5: Broken driver administration authorization

Baseline reproduction. An unauthenticated request changed Driver B's availability by using Driver B's URL identifier. See [evidence/V5-before.txt](#).

Impact. An attacker could disrupt delivery allocation and service availability.

Fix. Driver availability updates now require a valid driver JWT. The verified driver ID must match the URL driver ID.

Regression controls. One driver cannot update another driver's availability. A driver can update own availability. See [evidence/V5-after.txt](#), [security-tests/runtime-v4-v7.mjs](#), [security-tests/regression-v4-v7.mjs](#), commits [2f58bbf](#) and [8505d31](#).

Residual risk. This finding covers driver availability only. Restaurant administration requires separate review before it can be claimed as protected.

4.6 V6: Payment amount and state manipulation

Baseline reproduction. A client created a card order with `totalAmount=1` instead of the synthetic trusted total of 2500. The baseline also marked the order paid. See [evidence/V6-before.txt](#).

Impact. A customer could reduce the payable amount or obtain a paid order without confirmed payment.

Fix. The order service calculates the USD total from trusted product data. It ignores client currency and paid-status fields. New card orders start in the pending state.

Regression controls. Tampered amount, currency, and payment-state values do not control the persisted order. The test persists 2500 USD and pending state. An authenticated customer can create a server-priced order. See [evidence/V6-after.txt](#), [security-tests/runtime-v4-v7.mjs](#), [security-tests/regression-v4-v7.mjs](#), commits [2f58bbf](#) and [8505d31](#).

Residual risk. A future confirmation path must bind a trusted order to a verified payment-provider event.

4.7 V7: Sensitive token disclosure and session design

Baseline reproduction. Login and refresh responses returned bearer tokens in JSON. User-service logs recorded refresh-token data, and the browser stored access tokens. See [evidence/V7-before.txt](#).

Impact. Script injection, browser access, log access, or log forwarding could disclose bearer tokens and enable session impersonation.

Fix. Login and refresh now issue credentialed strict HttpOnly cookies. The frontend no longer stores bearer tokens. Request logging records the request path but omits query values, which also prevents OIDC authorization-code logging. Cookie mutations require the approved Origin.

Regression controls. Login and refresh JSON omit access tokens. Browser storage and application logs do not contain bearer tokens. A missing or wrong Origin blocks cookie mutation. An approved-origin session can access profile and refresh operations. See [evidence/V7-after.txt](#), [security-tests/runtime-v4-v7.mjs](#), [security-tests/regression-v4-v7.mjs](#), commits [2f58bbf](#) and [8505d31](#).

Residual risk. HttpOnly cookies reduce token theft but do not prevent a successful XSS attack from making same-origin requests while a session exists.

5 Verification and reproducibility

5.1 Fixtures and baseline integrity

The deterministic fixture definition is [security-tests/fixtures/security-fixtures.json](#). The reset command writes ignored active fixture data with local-only permissions. The team verified the fixture contents before runtime checks. Baseline attacks ran in a separate worktree checked out at the immutable tag.

5.2 Test evidence

Area	Before evidence	After evidence	Focused checks
V1–V3	evidence/V1-before.txt through V3-before.txt	evidence/V1-after.txt through V3-after.txt	security-tests/runtime-v1-v3.mjs
V4–V7	evidence/V4-before.txt through V7-before.txt	evidence/V4-after.txt through V7-after.txt	runtime-v4-v7.mjs and regression-v4-v7.mjs
OIDC	Not a baseline vulnerability	evidence/oidc-after.txt	regression-oidc.mjs and oidc-mocked.mjs under Node 20

The source-contract checks verify that the security controls remain in the changed routes and services. The runtime checks verify the HTTP behavior. The evidence files are redacted. They do not contain credentials, complete tokens, or cookies.

5.3 Supporting tools

The team ran [npmaudit](#) against user, order, payment, delivery, and driver services. The commands, tool versions, and severity counts are recorded in [evidence/tools/npm-audit.txt](#). These advisories are

dependency-maintenance input, not counted application vulnerabilities, because no endpoint impact was reproduced.

The OWASP ZAP local-gateway scan is deferred. The Docker image download and scan exceeded the available bandwidth. The repository records this honestly in [evidence/tools/zap-deferred.txt](#); no ZAP result is claimed. A bounded local-gateway scan remains a pre-submission action when bandwidth is available.

6 WSO2 OpenID Connect customer login

6.1 Feature objective

The added feature provides one customer sign-in path through WSO2 Identity Server 7.1.0. It does not add account linking, role synchronization, or automated logout. The intent is to demonstrate a secure OpenID Connect grant implementation without exposing provider tokens to the browser.

6.2 Security design

The user service creates cryptographically random state, nonce, and PKCE verifier values. It sends the S256 challenge to WSO2 and stores the temporary flow in a signed, short-lived, HttpOnly cookie. The callback compares state before token exchange, exchanges the authorization code server-side, and validates the provider issuer, client audience, RS256 signature, expiry, nonce, and subject.

The local CraveDrop account key is the provider issuer plus subject. The service explicitly rejects automatic local-account linking by matching an email address. The local WSO2 test user store released only a subject claim. In that controlled case, the service keeps the issuer-plus-subject binding and stores a deterministic non-deliverable [@identity.invalid](#) placeholder rather than assuming an email address.

6.3 Feature verification

A synthetic WSO2 customer completed a real local flow. The callback returned HTTP 302 to an allow-listed dashboard path, issued strict HttpOnly CraveDrop session cookies, expired the temporary flow cookie, and authorized a protected validation endpoint. No provider token appeared in the callback URL.

The mocked Node 20 suite confirms that a tampered state is rejected before token exchange and that an invalid nonce or identity token is rejected. The proof is [evidence/oidc-after.txt](#); the tests are [security-tests/regression-oidc.mjs](#) and [security-tests/oidc-mocked.mjs](#); the implementation commit is [4a440d1](#).

7 Remaining vulnerabilities and limitations

The following risks were identified but are not claimed as fixed or counted as additional vulnerabilities.

Risk or limitation	Reason not fixed in this scope	Recommended follow-up
Restaurant administration controls	The seven-findings scope uses driver availability for V5. Restaurant routes need an independent assessment and test set.	Review each restaurant mutation for verified role and ownership checks.

Risk or limitation	Reason not fixed in this scope	Recommended follow-up
Notification BOLA variants	These share an authorization root cause with other object-access findings. Counting them would inflate the seven-finding portfolio.	Assess them separately and add principal-scoped notification access.
XSS impact on cookie sessions	HttpOnly prevents token reads but not same-origin actions initiated by injected script.	Add a Content Security Policy, output encoding review, and XSS tests.
Payment confirmation integration	The local control demonstrates trusted order state, not a real payment-provider confirmation.	Verify signed provider webhooks and bind each event to the trusted order and amount.
Driver device and GPS integrity	API authorization cannot verify the physical source of a reported location.	Apply mobile/device controls and anomaly detection where appropriate.
ZAP scan	The planned scan is deferred because of bandwidth limits.	Run and triage a bounded local-gateway ZAP scan before submission.

8 Preventive secure software engineering practices

The observed vulnerabilities are preventable through routine engineering controls.

- **Threat modelling at route design.** Define the actor, resource owner, permitted action, and server-owned state before implementing a route.
- **Authorization as a service-boundary contract.** Verify identity from a signed token or session. Do not derive identity or privilege from body, query, or URL data alone.
- **Allow-list mutation inputs.** Accept only fields required by the user action. Compute prices, account IDs, payment state, and audit timestamps on the server.
- **Security regression tests.** Keep an attack test and an authorized control for every authorization and state-change rule.
- **Secure session engineering.** Keep bearer credentials out of JSON, browser storage, URLs, and logs. Use strict HttpOnly cookies with origin checks for cookie-mutating requests.
- **Dependency and configuration review.** Review audit output, container configuration, default credentials, logging, and environment handling. Do not treat an advisory as an exploit without impact evidence.
- **Evidence hygiene.** Use synthetic fixtures, redact captures, and ensure secret-bearing files are ignored before commits, reports, and videos.

9 Individual contributions

The repository records work in coherent commits. The final video must show each member explaining the allocated work. The team will not manufacture commits solely to increase contribution counts.

Member	Registration number	Allocated work and evidence
Hansaja A. M. G	IT22171856	User-session and token hardening, WSO2 OIDC login, 4a440d1 , evidence/oidc-after.txt , and OIDC video segment.
Dharmasiri I. D. N. D	IT22254320	Order authentication, BOLA, mass-assignment controls, 4904430 , 2e7d8bf , and V1–V3 video segments.
Sanjeewa P. D. L. B	IT22629708	Delivery and driver authorization, payment integrity, 2f58bbf , 8505d31 , and V4–V6 video segments.
Aluthwaththa A. W. D. S. M	IT22267290	Synthetic fixtures, 89421a7 , tool evidence, report material, README, submission packaging, and video segments.

10 Conclusion

CraveDrop now has seven independently evidenced security fixes and a secure WSO2 OIDC customer-login feature. The work preserves a reproducible vulnerable baseline and verifies both blocked attacks and legitimate authorized flows. The report states remaining risks and the deferred ZAP scan without unsupported claims. The final submission must add the unlisted video URL to [README.txt](#), run the remaining bounded scan if possible, and complete the clean-clone and ZIP checks.

References

- [1] SE4030, *Secure Software Development Assignment*, 2026. Provided assignment brief.
- [2] OWASP Foundation, *OWASP API Security Top 10*. <https://owasp.org/www-project-api-security/>
- [3] OWASP Foundation, *OWASP Top 10 Web Application Security Risks*. <https://owasp.org/www-project-top-ten/>
- [4] MITRE, *Common Weakness Enumeration*. <https://cwe.mitre.org/>
- [5] OpenID Foundation, *OpenID Connect Core 1.0*. https://openid.net/specs/openid-connect-core-1_0.html
- [6] IETF, *RFC 7636: Proof Key for Code Exchange by OAuth Public Clients*. <https://www.rfc-editor.org/rfc/rfc7636>
- [7] WSO2, *WSO2 Identity Server Documentation*. <https://is.docs.wso2.com/>